

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Charger le dataset

```
df = pd.read_csv("Crop_recommendation.csv")

df.head()
```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

Vérifier la structure des données

```
df.shape
```

```
(2200, 8)
```

```
df.info()
```

[Afficher la sortie masquée](#)

Vérifier les valeurs manquantes

```
df.isnull().sum()
```

[Afficher la sortie masquée](#)

Vérifier les doublons

```
df.duplicated().sum()
```

[Afficher la sortie masquée](#)

Analyse statistique descriptive

```
df.describe()
```

[Afficher la sortie masquée](#)

```
df['label'].unique()
```

[Afficher la sortie masquée](#)

```
plt.figure(figsize=(12,6))
sns.countplot(x=df['label'], order=df['label'].value_counts().index)
plt.xticks(rotation=90)
plt.title("Distribution of Crop Classes", fontsize=16)
plt.xlabel("Crop Type")
plt.ylabel("Count")
plt.show()
```

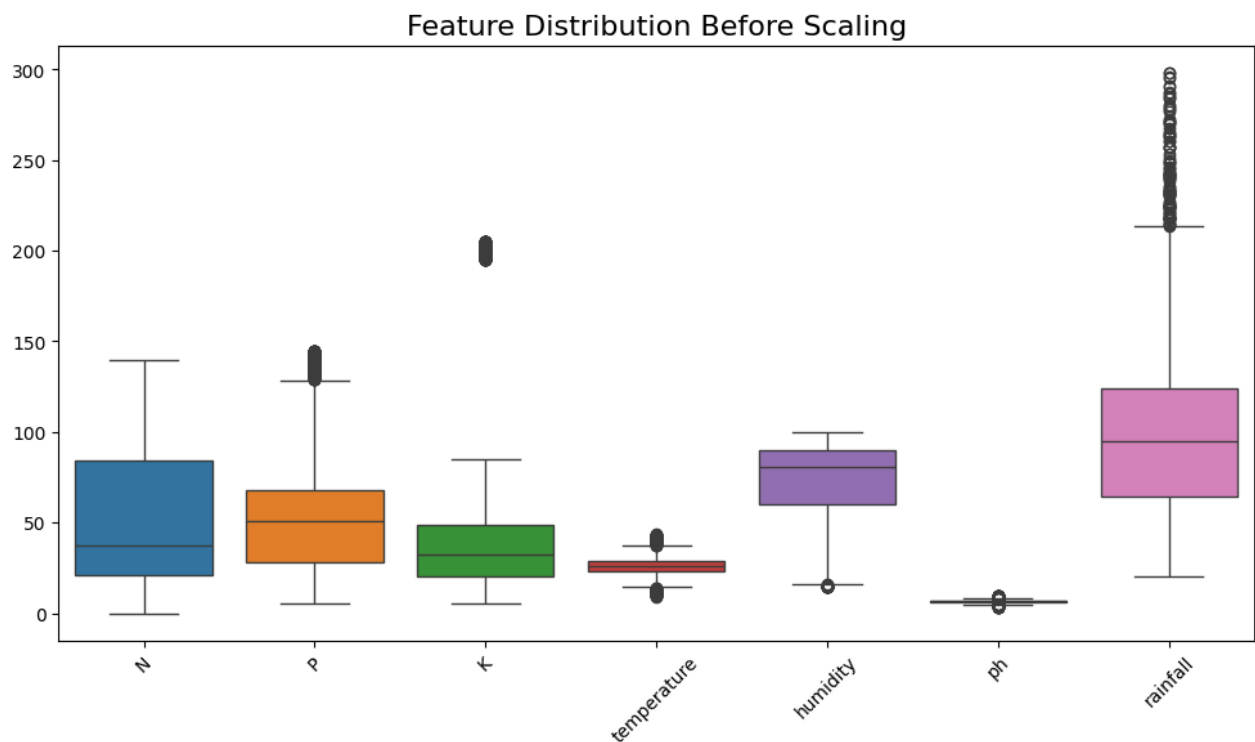
[Afficher la sortie masquée](#)

DATA PREPARATION

Séparer Features et Target

```
plt.figure(figsize=(12,6))
sns.boxplot(data=df.drop("label", axis=1))
```

```
plt.xticks(rotation=45)
plt.title("Feature Distribution Before Scaling", fontsize=16)
plt.show()
```



```
X = df.drop("label", axis=1)
y = df["label"]

X.head()
```

[Afficher la sortie masquée](#)

Encoder la variable cible (label)

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
y_encoded = le.fit_transform(y)

# Voir le mapping
mapping = dict(zip(le.classes_, le.transform(le.classes_)))
mapping
```

```
{'apple': np.int64(0),
 'banana': np.int64(1),
 'blackgram': np.int64(2),
 'chickpea': np.int64(3),
 'coconut': np.int64(4),
 'coffee': np.int64(5),
 'cotton': np.int64(6),
 'grapes': np.int64(7),
 'jute': np.int64(8),
 'kidneybeans': np.int64(9),
 'lentil': np.int64(10),
 'maize': np.int64(11),
 'mango': np.int64(12),
 'mothbeans': np.int64(13),
 'mungbean': np.int64(14),
 'muskmelon': np.int64(15),
 'orange': np.int64(16),
 'papaya': np.int64(17),
 'pigeonpeas': np.int64(18),
 'pomegranate': np.int64(19),
 'rice': np.int64(20),
 'watermelon': np.int64(21)}
```

Normalisation des données

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
X_scaled[:5]
```

```
array([[ 1.0687974, -0.34455075, -0.1016875, -0.93558693,  0.47266646,
         0.04330173,  1.8103605 ],
       [ 0.93332887,  0.14061552, -0.14118477, -0.75964603,  0.39705125,
         0.73487256,  2.24205791],
       [ 0.25598625,  0.04964684, -0.08193887, -0.51589814,  0.48695381,
         1.77151047,  2.92106603],
       [ 0.63529812, -0.55681099, -0.1609334,  0.17280704,  0.38980549,
         0.66030768,  2.53704803],
       [ 0.74367294, -0.34455075, -0.12143613, -1.08364706,  0.45479163,
         1.49786778,  2.89837267]])
```

Train / Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)
```

```
print("Train size:", X_train.shape)
print("Test size:", X_test.shape)
```

```
Train size: (1760, 7)
Test size: (440, 7)
```

Vérification

```
print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
```

```
X_train shape: (1760, 7)
y_train shape: (1760,)
```

XGBOOST

```
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

creation

```
xgb_model = XGBClassifier(
    objective="multi:softmax",
    num_class=len(le.classes_),
    n_estimators=200,
    learning_rate=0.1,
    max_depth=6,
    random_state=42
)
```

```
xgb_model.fit(X_train, y_train)
```

```

XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric=None, feature_types=None,
               feature_weights=None, gamma=None, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=0.1, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=6,
               max_leaves=None, min_child_weight=None, missing=nan,
               monotone_constraints=None, multi_strategy=None, n_estimators=200,
               n_jobs=None, num_class=22, ...)

```

```
y_pred = xgb_model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Accuracy: 0.9886363636363636

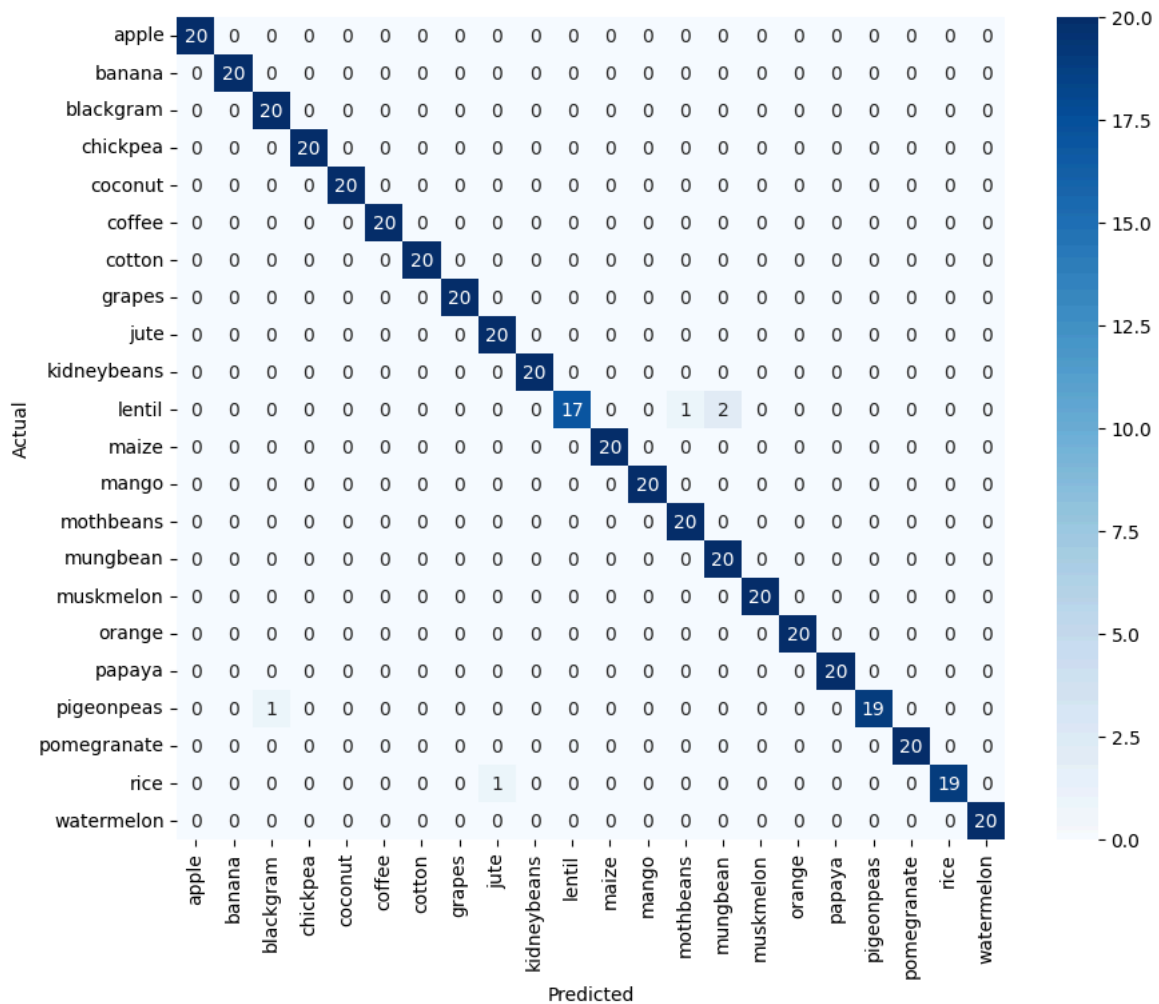
```
print(classification_report(y_test, y_pred, target_names=le.classes_))
```

	precision	recall	f1-score	support
apple	1.00	1.00	1.00	20
banana	1.00	1.00	1.00	20
blackgram	0.95	1.00	0.98	20
chickpea	1.00	1.00	1.00	20
coconut	1.00	1.00	1.00	20
coffee	1.00	1.00	1.00	20
cotton	1.00	1.00	1.00	20
grapes	1.00	1.00	1.00	20
jute	0.95	1.00	0.98	20
kidneybeans	1.00	1.00	1.00	20
lentil	1.00	0.85	0.92	20
maize	1.00	1.00	1.00	20
mango	1.00	1.00	1.00	20
mothbeans	0.95	1.00	0.98	20
mungbean	0.91	1.00	0.95	20
muskmelon	1.00	1.00	1.00	20
orange	1.00	1.00	1.00	20
papaya	1.00	1.00	1.00	20
pigeonpeas	1.00	0.95	0.97	20
pomegranate	1.00	1.00	1.00	20
rice	1.00	0.95	0.97	20
watermelon	1.00	1.00	1.00	20
accuracy			0.99	440
macro avg	0.99	0.99	0.99	440
weighted avg	0.99	0.99	0.99	440

```
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=le.classes_,
            yticklabels=le.classes_)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Random Forest

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,
    random_state=42
)
```

```
rf_model.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(n_estimators=200, random_state=42)
```

```
y_pred_rf = rf_model.predict(X_test)
```

```
accuracy_rf = accuracy_score(y_test, y_pred_rf)
print("Random Forest Accuracy:", accuracy_rf)
```

```
Random Forest Accuracy: 0.9954545454545455
```

```
print(classification_report(y_test, y_pred_rf, target_names=le.classes_))
```

	precision	recall	f1-score	support
apple	1.00	1.00	1.00	20
banana	1.00	1.00	1.00	20
blackgram	1.00	0.95	0.97	20
chickpea	1.00	1.00	1.00	20
coconut	1.00	1.00	1.00	20
coffee	1.00	1.00	1.00	20
cotton	1.00	1.00	1.00	20
grapes	1.00	1.00	1.00	20
jute	0.95	1.00	0.98	20

kidneybeans	1.00	1.00	1.00	20
lentil	1.00	1.00	1.00	20
maize	0.95	1.00	0.98	20
mango	1.00	1.00	1.00	20
mothbeans	1.00	1.00	1.00	20
mungbean	1.00	1.00	1.00	20
muskmelon	1.00	1.00	1.00	20
orange	1.00	1.00	1.00	20
papaya	1.00	1.00	1.00	20
pigeonpeas	1.00	1.00	1.00	20
pomegranate	1.00	1.00	1.00	20
rice	1.00	0.95	0.97	20
watermelon	1.00	1.00	1.00	20
accuracy			1.00	440
macro avg	1.00	1.00	1.00	440
weighted avg	1.00	1.00	1.00	440

```
import seaborn as sns
import matplotlib.pyplot as plt

cm_rf = confusion_matrix(y_test, y_pred_rf)

plt.figure(figsize=(10,8))
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Greens',
            xticklabels=le.classes_,
            yticklabels=le.classes_)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Random Forest")
plt.show()
```

